Lineare und Mixed-Integer Programmierung

Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de , Raum 149B
Version: 1.0.5.1

Eine allererste Einführung[1]

Folien: https://delors.github.io/theo-algo-mixed_integer_programming/folien.de.md.html
https://delors.github.io/theo-algo-mixed_integer_programming/folien.de.md.html.pdf
Fehler melden: https://github.com/Delors/delors.github.io/issues
KI Verwendung: Bei der Erstellung der Unterlagen wurden KI Assistenten (insbesondere Claude aber ggf. auch ChatGPT, Ollama mit Gemma/LLama/Qwen oder OpenCode mit Kimi/Qwen/Deepseek...) unterstützend eingesetzt. Dies erfolgte insbesondere zur Unterstützung bei der Generierung von Grafiken (d. h. SVG Dateien), oder um sich Übersichtstabellen generieren zu lassen. Weiterhin wurde KI zur allgemeinen Qualitätssicherung eingesetzt. Inhalte, die ggf. von der KI vorgeschlagen wurden, wurden im Falle der Übernahme explizit validiert und angepasst.

---

[1] Introduction to Algorithms, 3rd Edition, Cormen, Leiserson, Rivest, Stein, MIT Press, 2009

1. Einführung in lineare Programmierung

Beispielszenario: Kostenoptimierung

Optimierung der Kosten für die Nahrungsmittelzusammensetzung

Seien x_1 und x_2 die Menge an Nahrungsmitteln 1 und 2, die wir kaufen. Die Kosten für Nahrungsmittel 1 und 2 betragen 1 und 2 Euro pro Einheit. Die täglichen Ernährungsbedürfnisse sind 10 Einheiten Protein und 20 Einheiten Fett. Nahrungsmittel 1 enthält 2 Einheiten Protein und 3 Einheiten Fett pro Einheit. Nahrungsmittel 2 enthält 1 Einheit Protein und 4 Einheiten Fett pro Einheit.

Zielfunktion (🇺🇸 *objective function* oder einfach nur 🇺🇸 *objective*)

$$\text{minimiere } x_1 \cdot 1€ + x_2 \cdot 2€$$

(unter den) Nebenbedingungen (🇺🇸 *constraints*/🇺🇸 *subject to (s.t.)*)

$$\begin{aligned} 2 \cdot x_1 + 1 \cdot x_2 &\geq 10 && \text{Nebenbedingung bzgl. Protein} \\ 3 \cdot x_1 + 4 \cdot x_2 &\geq 20 && \text{Nebenbedingung bzgl. Fett} \\ x_1, x_2 &\geq 0 \end{aligned}$$

Lineare Programmierung

Definition

Lineare Programmierung: Optimierung von linearen Funktionen unter linearen Nebenbedingungen.

Das Ziel ist die Optimierung (Maximierung/Minimierung) einer linearen Funktion f:

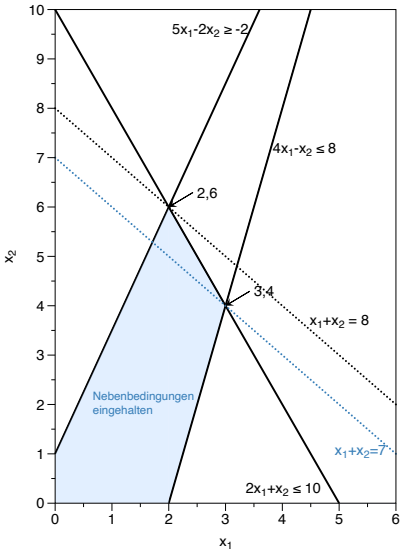
$$f(x_1, \ldots, x_n) = a_1 \cdot x_1 + a_2 \cdot x_2 + \ldots + a_n \cdot x_n = \sum_{i=1}^{n} a_i \cdot x_i$$

Unter einer Menge von linearen Nebenbedingungen. Sei $b \in \mathbb{R}$, dann ist ...

- eine *lineare Ungleichung* der Form: $f(x_1, \ldots, x_n) \leq b$
- eine *lineare Gleichung* der Form: $f(x_1, \ldots, x_n) = b$
- lineare Ungleichungen und Gleichungen beschreiben die *linearen Nebenbedingungen*.

Lösen von linearen Optimierungsproblemen

Standardform - „nur“ Verwendung von linearen Ungleichungen



Zielfunktion (Maximiere)

$$x_1 + x_2$$

Nebenbedingungen

$$\begin{array}{rcrcr} 4x_1 & - & x_2 & \leq & 8 \\ 2x_1 & + & x_2 & \leq & 10 \\ 5x_1 & - & 2x_2 & \geq & -2 \\ & & x_1 & \geq & 0 \\ & & x_2 & \geq & 0 \end{array}$$

Schlupfform (🇺🇸 *Slack Form*) - „nur“ Verwendung von linearen Gleichungen

Zielfunktion (Maximiere)

$$x_1 + x_2$$

unter den Nebenbedingungen

$$\begin{array}{rcrcrcr} x_3 & = & 8 & - & 4x_1 & + & x_2 \\ x_4 & = & 10 & - & 2x_1 & - & x_2 \\ x_5 & = & 2 & + & 5x_1 & - & 2x_2 \\ 0 & \leq & x_1, & & x_2, & & x_3, & & x_4, & & x_5 \end{array}$$

Die Variablen x_3, x_4 und x_5 sind die Schlupfvariablen. Sie messen die Differenz zwischen der linken und der rechten Seite der Ungleichungen und sind nicht Teil der Zielfunktion.

Beobachtungen (am Beispiel orientiert)

- der Bereich der zulässigen Lösungen enthält (im Allgemeinen) unendlich viele Punkte
- der Bereich der zulässigen Lösungen ist beschränkt/ist (hier) ein konvexes Polygon (im Allgemeinen ein konvexes Polyeder)
- Die konvexe Hülle einer endlichen Anzahl von affin unabhängigen Punkten in einem n-dimensionalen Raum bezeichnen wir als Simplex
- in diesem (2-Dimensionalen) Fall können wir die Lösung grafisch darstellen
- nicht jedes lineare Optimierungsproblem hat eine (bzw. eine optimale) Lösung
- Auch in der Schlupfform, werden die Anforderungen an die nicht-Negativität der Variablen als Ungleichungen beschrieben.

Affine Unabhängigkeit:

Zwei Punkte sind affin unabhängig, wenn die Differenz der beiden Punkte nicht durch einen Skalarfaktor auf den anderen Punkt abgebildet werden kann. (Im 2-D Raum: Die beiden Punkte liegen nicht auf einer Geraden, wenn die beiden Punkte als entsprechende Vektoren aufgefasst werden.)

Die Schlupfform ist für den Simplex-Algorithmus relevant.

✎ Übung[2]

1.1. Formulierung eines linearen Programms

In einem Betrieb mit mehrschichtiger Arbeitszeit besteht folgender Mindestbedarf an Personal:

```
von 0 bis 4 Uhr: 3 Personen
von 4 bis 8 Uhr: 8 Personen
von 8 bis 12 Uhr: 10 Personen
von 12 bis 16 Uhr: 8 Personen
von 16 bis 20 Uhr: 14 Personen
von 20 bis 24 Uhr: 5 Personen
```

Der Arbeitsbeginn ist jeweils um 0, 4, 8, 12, 16 bzw. 20 Uhr. Die Arbeitszeit beträgt stets 8 Stunden hintereinander. Formulieren Sie ein lineares Program, um einen Einsatzplan mit minimalem Gesamtpersonalbedarf aufzustellen.

---

[2] Aus: Übungsbuch Operations Research; Domschke, Drexl, Schildt, Scholl, Voß; Springer Verlag 1997

✎ Übung

1.2. Berechnung des maximalen Flusses (Maximum-Flow-Problem)

Formulieren Sie ein lineares Programm zur Bestimmung des maximalen Flusses von einer Quelle s zu einer Senke t in einem Netzwerk mit V Knoten. (Die Funktion) f_{uv} sei der Fluss zwischen zwei Knoten u und v. Nebenbedingungen:

Kapazitätsbeschränkung:
Der Fluss f_{uv} auf einer Kante darf die Kapazität ($c(u, v)$) der Kante nicht überschreiten.

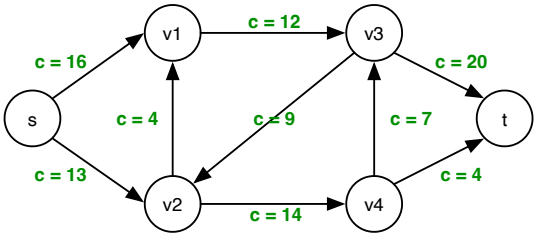
Flusserhaltung: Für jeden Knoten (außer Quelle und Senke) gilt, dass der zufließende Fluss gleich dem abfließenden Fluss ist.

Richtungsabhängigkeit:
Der Fluss ist gerichtet (von einem Knoten zum anderen).

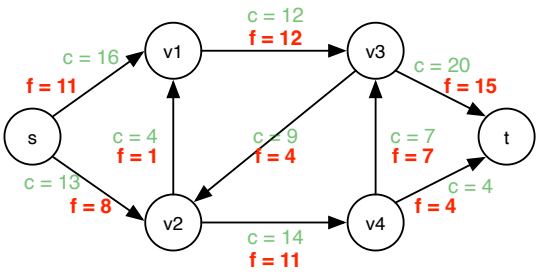
Sie können die vereinfachende Annahme machen, dass die Summe der Zuflüsse zur Quelle 0 ist ($\sum_{v \in V} f_{vs} = 0$); dass die Quelle keine eingehenden Kanten hat. Weiterhin sei die Kapazität zwischen zwei nicht-verbundenen Knoten 0.

Beispiel

Netzwerk mit Kapazitäten:



Eine (nicht notwendigerweise optimale) Lösung, die die Nebenbedingungen erfüllt:



Im Allgemeinen gilt

Das Netzwerk ist modelliert als gerichteter Graph $G = (V, E)$ ohne Eigenschleifen und ohne antiparallele Kanten (d. h. $(v, u) \in E \Rightarrow (u, v) \notin E$). Jeder Kante $(u, v) \in E$ ist eine nicht-negative Kapazität $c(u, v) \geq 0$ zugeordnet.

Sei $(u, v) \notin E$, dann ist $c(u, v) = 0$.

Empfohlene Vorgehensweise

1. Bestimmen Sie die Zielfunktion in Hinblick auf den Fluss bzw. der Variablen, die den Fluss repräsentieren.

2. Formulieren Sie die Nebenbedingungen:
    1. in Hinblick darauf, dass der Fluss über eine Kante nie negativ sein darf
    2. in Bezug auf die Kanten und die Kapazitäten
    3. in Bezug auf die Kapazitätserhaltung

2. Simplex Algorithmus

Simplex Algorithmus - Einführung

- Der Simplex-Algorithmus ist ein Algorithmus zur Lösung von linearen Optimierungsproblemen.
- Der Algorithmus wurde von George Dantzig entwickelt und 1947 veröffentlicht.
- Der Simplex-Algorithmus ist ein iteratives Verfahren, das in der Regel sehr effizient ist

  Im Regelfall polynomielle Laufzeit, im Worst-case jedoch exponentiell.
- Der Simplex-Algorithmus ist ein Beispiel für einen Algorithmus, der auf einem Netzwerk von Kanten operiert.

  Der Simplex-Algorithmus bewegt sich systematisch entlang der Ecken (🇺🇸 *Vertices*) des Bereichs, der die zulässigen Lösungen des linearen Programms beschreibt, um die optimale Lösung zu finden. Er terminiert, wenn er das lokale Optimum erreicht hat. Aufgrund der konvexen Natur des Problems ist das lokale Optimum gleichzeitig das globale Optimum.

Es gibt Algorithmen, die eine garantierte polynomielle Laufzeit haben, wie zum Beispiel der Ellipsoid-Algorithmus. Der Simplex-Algorithmus ist jedoch in der Praxis oft schneller.

Standardform

Gegeben sein n reelle Zahlen $(c_1, ..., c_n)$; m reelle Zahlen ($b_1, ..., b_m$); und eine $m \times n$ Matrix $A = (a_{ij})$ für $i = 1, 2, ...m$ und $j = 1, 2, ...n$.

Wir möchten nun n reelle Zahlen $(x_1, ..., x_n)$ finden, die die folgenden Bedingungen erfüllen:

Zielfunktion (🇺🇸 *objective function*)

$$\text{maximiere} \sum_{j=1}^{n} c_j \cdot x_j$$

(unter den) Nebenbedingungen (🇺🇸 *subject to/constraints*)

$$\begin{array}{rcll} \sum_{j=1}^{n} a_{ij} \cdot x_j & \leq & b_i & \text{für } i = 1, 2, ..., m \\ x_j & \geq & 0 & \text{für } j = 1, 2, ..., n \end{array}$$

Kompakte Darstellung

Gegeben Matrix $A = (a_{ij})$, m-Vektor $b = (b_i)$, n-Vektor $c = (c_j)$, und n-Vektor $x = (x_j)$. Dann ist das lineare Programm:

$$\begin{array}{rcll} \text{maximiere} & c^T x & & \\ \text{unter den Nebenbedingungen} & A \cdot x & \leq & b \\ & x & \geq & 0 \end{array}$$

Terminologie

zulässige Lösung/🇺🇸 *feasible*:
: Eine Belegung der Variablen $\bar{x}$, die die Nebenbedingungen erfüllt.

optimale Lösung:
: Eine zulässige Lösung, die die Zielfunktion maximiert.

unbeschränkt:
: Ein lineares Programm, das Lösungen hat, die die Zielfunktion nicht beschränken.

unzulässig/🇺🇸 *infeasible*:
: Ein lineares Programm, das keine zulässige Lösung hat.

Konvertierung von beliebigen linearen Programmen in die Standardform

Ein lineares Programm kann aus folgenden vier Gründen nicht in Standardform sein:

- Die Zielfunktion ist zu minimieren
- Es gibt Variablen ohne Nichtnegativitätsbedingung
- Es gibt Gleichungen ($=$)
- Es gibt Ungleichungen mit $\geq$ statt $\leq$

Regeln

1. Minimierungsprobleme können durch Multiplikation der Zielfunktion mit -1 in ein Maximierungsproblem umgewandelt werden.
2. Variablen ohne Nichtnegativitätsbedingung können durch die Einführung von Differenzvariablen in Nichtnegativitätsbedingungen umgewandelt werden.

D. h. wir ersetzen die Vorkommen der Variablen $\boldsymbol{c} \cdot \boldsymbol{x}$ durch $\boldsymbol{c} \cdot \boldsymbol{x}^{+} - \boldsymbol{c} \cdot \boldsymbol{x}^{-}$ wobei $\boldsymbol{x}^{+}$ und $\boldsymbol{x}^{-}$ nicht-negativ sind.

3. Gleichungen können in zwei Ungleichungen umgewandelt werden.
4. Ungleichungen mit $\geq$ können durch Multiplikation mit -1 in Ungleichungen mit $\leq$ umgewandelt werden.

- $\boldsymbol{c}^{T}\boldsymbol{x}$ ist das innere Produkt.
- $\boldsymbol{x} \geq \boldsymbol{0}$ bedeutet, dass jede Komponente von $\boldsymbol{x}$ nicht negativ sein darf.

✎ Übung

2.1. Überführen Sie das lineare Programm in Standardform.

$$
\begin{array}{rrrrrr}
\text{minimiere} & -2x_1 & + & 3x_2 & & \\
\text{unter den Nebenbedingungen} & x_1 & + & x_2 & = & 7 \\
& x_1 & - & 2x_2 & \leq & 4 \\
& x_1 & & & \geq & 0
\end{array}
$$

2.2. Zeigen Sie, dass das folgende lineare Programm unzulässig ist.

$$
\begin{array}{rrrrrr}
\text{maximiere} & 3x_1 & - & 2x_2 & & \\
\text{unter den Nebenbedingungen} & x_1 & + & x_2 & \leq & 2 \\
& -2x_1 & - & 2x_2 & \leq & -10 \\
& & & x_1 & \geq & 0 \\
& & & x_2 & \geq & 0
\end{array}
$$

Schlupfform (🇺🇸 *Slack Form*)

- Zur effizienten Lösung von linearen Programmen wird die Schlupfform verwendet.
- Bei der Schlupfform werden alle Nebenbedingungen in Gleichungen umgewandelt - abgesehen von den Nichtnegativitätsbedingungen.

Vorgehen

Gegeben sei *eine* Ungleichung:

$$\sum_{j=1}^{n} a_{ij} \cdot x_j \leq b_i \tag{1}$$

Wir führen dann eine Schlupfvariable (🇺🇸 *slack variable*) x_{n+i} ein und erhalten:

$$x_{n+1} = b_i - \sum_{j=1}^{n} a_{ij} \cdot x_j \tag{2}$$

$$x_{n+1} \geq 0 \tag{3}$$

- Somit stehen die Variablen $x_1, ..., x_n$ für die ursprünglichen Variablen und die Variablen $x_{n+1}, ..., x_{n+m}$ für die Schlupfvariablen.
- Auf der rechten Seite der Gleichung (2) stehen die ursprünglichen Variablen. Nur diese Variablen sind (initial) Teil der Zielfunktion.

Basisvariablen: Die Variablen auf der linken Seite der Gleichung (2).

Nichtbasisvariablen: Die Variablen auf der rechten Seite der Gleichung (2).

- Für den Wert der Zielfunktion verwenden wir die Variable z.
- Im Folgenden gilt:

N:	die Menge der Indizes der Nichtbasisvariablen
B:	die Menge der Indizes der Basisvariablen
v:	ein optionaler konstanter Term in der Zielfunktion

$|N| = n$, $|B| = m$ und $N \cup B = 1, ..., n + m$

Somit ist die kompakte Darstellung des linearen Programms in Schlupfform:

$$\begin{aligned} z &= v + \sum_{j \in N} c_j \cdot x_j \\ x_i &= b_i - \sum_{j \in N} a_{ij} \cdot x_j \quad \text{für } i \in B \end{aligned}$$

Diese Schlupfvariable (x_{n+1}) misst die Differenz zwischen der linken und der rechten Seite der Ungleichung (1).

Schlupfform - Beispiel

Gegebenes lineares Programm:

$$\begin{array}{rrcrcrcl} \text{maximiere} & 3x_1 & + & x_2 & + & 2x_3 & & \\ \text{unter den Nebenbedingungen} & x_1 & + & x_2 & + & 3x_3 & \leq & 30 \\ & 2x_1 & + & 2x_2 & + & 5x_3 & \leq & 24 \\ & 4x_1 & + & x_2 & + & 2x_3 & \leq & 36 \\ & x_1 & , & x_2 & , & x_3 & \geq & 0 \end{array}$$

Gegebenes lineares Programm in Schlupfform:

Wir führen die Schlupfvariablen x_4, x_5 und x_6 ein mit der Nebenbedingung: $x_4, x_5, x_6 \geq 0$.

$$\begin{array}{rrcrcrcrcl} \text{maximiere} & z & = & & & 3x_1 & + & x_2 & + & 2x_3 \\ \text{unter den Nebenbedingungen} & x_4 & = & 30 & - & x_1 & - & x_2 & - & 3x_3 \\ & x_5 & = & 24 & - & 2x_1 & - & 2x_2 & - & 5x_3 \\ & x_6 & = & 36 & - & 4x_1 & - & x_2 & - & 2x_3 \end{array}$$

✎ Übung

2.3. Überführen eines linearen Programms in Schlupfform

Überführen Sie das 1. lineare Programm aus der vorhergehenden Übung in Schlupfform.

Bauen Sie ggf. auf den Ergebnissen der vorhergehenden Aufgabe auf.

2.4. Zeigen Sie (grafisch), das das folgende lineare Programm unbeschränkt ist.

$$
\begin{array}{rrcrcl}
\text{maximiere} & x_1 & - & x_2 & & \\
\text{unter den Nebenbedingungen} & -2x_1 & + & x_2 & \leq & -1 \\
& -x_1 & - & 2x_2 & \leq & -2 \\
& x_1, & x_2 & & \geq & 0
\end{array}
$$

(Primaler) Simplex

Grundlegende Idee

Wir lösen unser Optimierungsproblem durch gezielte algebraische Operationen, die die Zielfunktion maximieren.

1. Wir wählen immer eine Variable, die in der Zielfunktion vorkommt und einen positiven Koeffizienten hat.

   (D. h. wir wählen eine Variable deren Erhöhung die Zielfunktion maximiert.)
2. Dann bestimmen wir die Ungleichung, die die Maximierung der gewählten Variable am stärksten einschränkt.
3. Wir „tauschen“ die Variable mit der Schlupfvariablen, die in dieser Ungleichung vorkommt und lösen die Gleichung nach der gewählten Variable auf.
4. Wir setzen dann die umgestellte Gleichung in alle anderen Gleichungen (inkl. Zielfunktion) ein, um die Werte der anderen Variablen zu bestimmen.

Wir nennen diesen Prozess (1-4) „Pivot Operation“.
Wir wiederholen diesen Prozess, bis wir keine Variable mehr finden, die die Zielfunktion maximiert.
An dieser Stelle können wir dann das Optimum und die Werte für die Variablen ($x_i, \ldots, x_{n+m}$) ablesen.

Es ist in Hinblick auf die Korrektheit gleichgültig welche Variable wir im ersten Schritt wählen. Es kommt jedoch ggf. zu einer unterschiedlichen Anzahl an Schritten, bis wir die optimale Lösung finden.

Simplex anwenden

Gegebenes lineares Programm in Schlupfform

Wir führen die Schlupfvariablen x_4, x_5 und x_6 ein mit der Nebenbedingung: $x_4, x_5, x_6 \geq 0$.

$$\begin{array}{rrcrcrcrcr} \textbf{maximiere} & z & = & & & 3x_1 & + & x_2 & + & 2x_3 \\ \textbf{unter den Nebenbedingungen} & x_4 & = & 30 & - & x_1 & - & x_2 & - & 3x_3 \\ & x_5 & = & 24 & - & 2x_1 & - & 2x_2 & - & 5x_3 \\ & x_6 & = & 36 & - & 4x_1 & - & x_2 & - & 2x_3 \end{array}$$

- Wir können die Zielfunktion maximieren, indem wir die Variable der Zielfunktion mit dem größten positiven Koeffizienten wählen: x_1.
- Wir prüfen welche Nebenbed. die Maximierung von x_1 am stärksten einschränkt:

  1. Nebenbed.: $x_1 \leq 30$, 2. Nebenbed.: $x_1 \leq 12$ und 3. Nebenbed.: $x_1 \leq 9$

- Die (nicht-Basis)Variable x_1 wird somit durch die Schlupfvariable/Basisvariable x_6 ersetzt:

$$4x_1 = 36 - x_6 - x_2 - 2x_3 \Rightarrow x_1 = 9 - \frac{1}{4}x_6 - \frac{1}{4}x_2 - \frac{1}{2}x_3$$

- Wir setzen x_1 in die Zielfunktion und die anderen Nebenbedingungen ein und erhalten:

  $x_4 = 30 - (9 - \frac{1}{4}x_6 - \frac{1}{4}x_2 - \frac{1}{2}x_3) - x_2 - 3x_3$

  $x_4 = 21 + \frac{1}{4}x_6 - \frac{3}{4}x_2 - \frac{5}{2}x_3$

- Ergebnis

$$\begin{array}{rcrcrcrcr} z & = & 27 & - & \frac{3}{4}x_6 & + & \frac{1}{4}x_2 & + & \frac{1}{2}x_3 \\ x_1 & = & 9 & - & \frac{1}{4}x_6 & - & \frac{1}{4}x_2 & - & \frac{1}{2}x_3 \\ x_4 & = & 21 & + & \frac{1}{4}x_6 & - & \frac{3}{4}x_2 & - & \frac{5}{2}x_3 \\ x_5 & = & 6 & + & \frac{1}{2}x_6 & - & \frac{3}{2}x_2 & - & 4x_3 \end{array}$$

  Diese Operation wird als Pivot Operation bezeichnet.

- Im nächsten Schritt könnten wir x_3 wählen, da es den größten positiven Koeffizienten hat. Da die dritte Nebenbedingung die Maximierung von x_3 am stärksten einschränkt, würden wir x_3 durch die Schlupfvariable x_5 ersetzen.
- Ergebnis

$$\begin{array}{rcrcrcrcr} z & = & \frac{111}{4} & + & \frac{1}{16}x_2 & - & \frac{1}{8}x_5 & - & \frac{11}{16}x_6 \\ x_1 & = & \frac{33}{4} & - & \frac{1}{16}x_2 & + & \frac{1}{8}x_5 & - & \frac{5}{16}x_6 \\ x_3 & = & \frac{3}{2} & - & \frac{3}{8}x_2 & - & \frac{1}{4}x_5 & + & \frac{1}{8}x_6 \\ x_4 & = & \frac{69}{4} & + & \frac{3}{16}x_2 & + & \frac{5}{8}x_5 & - & \frac{1}{16}x_6 \end{array}$$

- Die Basislösung ist: $(33/4, 0, 3/2, 69/4, 0, 0)$ und der Wert der Zielfunktion ist $111/4$.
- Im letzten Schritte würden wir x_2 wählen. Da die zweite Nebenbedingung die Maximierung von x_2 am stärksten einschränkt, würden wir x_2 durch die Schlupfvariable x_3 ersetzen.
- Ergebnis

$$
\begin{array}{rcrcrcrcr}
z & = & 28 & - & \frac{1}{6}x_3 & - & \frac{1}{6}x_5 & - & \frac{2}{3}x_6 \\
x_1 & = & 8 & + & \frac{1}{6}x_3 & + & \frac{1}{6}x_5 & - & \frac{1}{3}x_6 \\
x_2 & = & 4 & - & \frac{8}{3}x_3 & - & \frac{2}{3}x_5 & + & \frac{1}{3}x_6 \\
x_4 & = & 18 & + & \frac{1}{2}x_3 & + & \frac{1}{2}x_5 & &
\end{array}
$$

- Die Basislösung ist: $(8, 4, 0, 18, 0, 0)$ und der Wert der Zielfunktion ist 28.
- Eine weitere Verbesserung der Zielfunktion ist nicht möglich. Die Basislösung ist somit unsere optimale Lösung.

Beobachtungen:

- Beim Start: jede Belegung der Variablen $x_1, ..., x_3$ definiert Werte für die Variablen $x_4, ..., x_6$ und ist somit eine Lösung.
- eine Lösung ist (jedoch nur) dann zulässig wenn alle Variablen nicht-negativ sind.
- Die Basislösung ist die Lösung, bei der die nicht-Basisvariablen (im ersten Schritt also x_1, x_2 und x_3) den Wert 0 haben. Im ersten Schritt ergibt sich somit die Basislösung ($\bar{x_1}, ..., \bar{x_6}$) $(0, 0, 0, 30, 24, 36)$; der Wert der Zielfunktion ist 0.

Simplex Algorithmus

```
Algorithm Simplex(A,b,c):
    (N,B,A,b,c,v) := InitialisiereSimplex(A,b,c)
    sei Δ ein Vektor der Länge m
    while ∃ j ∈ N mit c_j > 0 do
        wähle Index e ∈ N mit c_e > 0   { e für "entering variable" }
        for Index i ∈ B
            Δ_i := b_i / A_ie falls A_ie > 0, sonst ∞
        wähle l ∈ B mit Δ_l := min(Δ_1,...,Δ_m)
        if Δ_l = ∞ then return "unbeschränkt"
        (N,B,A,b,c,v) := Pivot(N,B,A,b,c,v,l,e)
    for i := 1 to n                    { Gib die Lösung zurück }
        if i ∈ B then
            x_i := b_i
        else
            x_i := 0
    return (x_1,...,x_n)
```

InitialisiereSimplex(A,b,c)

Falls das LP lösbar ist, dann gib das LP in Schlupfform zurück, in der die initiale Basislösung zulässig ist.

Wir werden uns im Rahmen dieses Kurses nicht weiter mit der Implementierung des Simplex-Algorithmus beschäftigen. Es ist jedoch wichtig, dass Sie die Funktionsweise des Algorithmus verstehen.

✎ Übung

2.5. Anwenden des Simplex-Algorithmus

Berechnen Sie die optimale Lösung für das folgende lineare Programm:

$$
\begin{array}{rrcrcl}
\textbf{maximiere} & 40x_1 & + & 30x_2 & & \\
\textbf{unter den Nebenbedingungen} & x_1 & + & x_2 & \leq & 8 \\
& 2x_1 & + & x_2 & \leq & 12 \\
& 2x_1 & + & 3x_2 & \leq & 18 \\
& x_1, & x_2 & & \geq & 0
\end{array}
$$

3. Mixed-Integer-Programmierung (MIP)

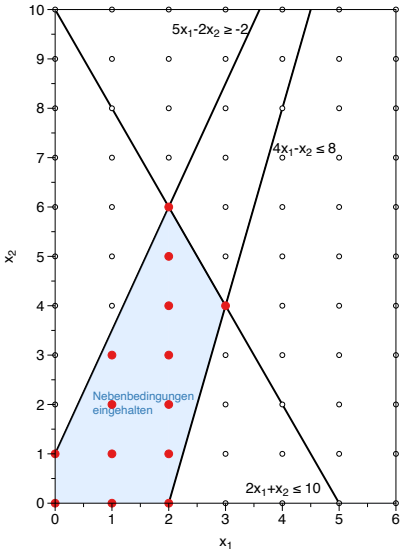
MIP: einige (oder alle) Variablen sind ganzzahlig

Zielfunktion (Maximiere)

$$x_1 + x_2$$

Nebenbedingungen

$$\begin{array}{rcrcll} 4x_1 & - & x_2 & \leq & 8 & \\ 2x_1 & + & x_2 & \leq & 10 & \\ 5x_1 & - & 2x_2 & \geq & -2 & \\ & & x_1 & \geq & 0 & \text{und ganzzahlig} \\ & & x_2 & \geq & 0 & \text{und ganzzahlig} \end{array}$$

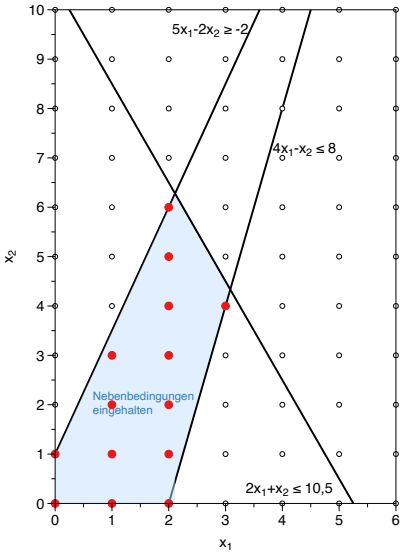


Zielfunktion (Maximiere)

$$x_1 + x_2$$

Nebenbedingungen

$$\begin{array}{rcrcll} 4x_1 & - & x_2 & \leq & 8 & \\ 2x_1 & + & x_2 & \leq & 10,5 & \\ 5x_1 & - & 2x_2 & \geq & -2 & \\ & & x_1 & \geq & 0 & \text{und ganzzahlig} \\ & & x_2 & \geq & 0 & \text{und ganzzahlig} \end{array}$$



Hinweis

Durch die Einschränkung, dass die Variablen ganzzahlig sein müssen, wird das Problem schwieriger zu lösen und ist NP-schwer, während das lineare Programm in polynomieller Zeit gelöst werden kann.

Bemerkung

Zur Lösung von MIPs gibt es verschiedene Ansätze, wie z. B. den Branch-and-Bound-Algorithmus, bzw. Branch-and-Cut-Algorithmus. Häufig werden in der Praxis auch Kombinationen von Algorithmen eingesetzt, die auf dem Simplex-Algorithmus basieren.

Wenn alle Variablen ganzzahlig sind, sprechen wir von einem reinen ganzzahligen Programm (🇺🇸 *Integer Programming*). Wenn nur einige Variablen ganzzahlig sind, sprechen wir von einem gemischt ganzzahligem Programm.

◆ Bemerkung

Wir konzentrieren uns im Folgenden darauf für konkrete Probleme, ganzzahlige Programme zu entwickeln. Wir betrachten die zugrunde liegenden Algorithmen nicht.

Binärvariablen oder ganzzahlige Variablen?

Sudokus lösen

8			6			9		5
				2		3	1	
		7	3	1	8		6	
2	4						7	3
		2	7	9		1		
5				8			3	6
		3						

Naiver Ansatz

Wir verwenden 81 Integer Variablen $1 \leq y_i \leq 9$.

Und jetzt?

Faustregel

- Verwenden Sie allgemeine Ganzzahlen (Integers), wenn sie tatsächliche Mengen darstellen und die Reihenfolge wichtig ist!
- Verwenden Sie Binärzahlen ($\{0, 1\}$ für jeden möglichen Wert einer Ganzzahl), wenn die Ganzzahlen konzeptuell nur „einige verschiedene Werte“ darstellen!

Beispiel: SEND + MORE = MONEY mittels Integer Programmierung

Problembeschreibung: SEND+MORE=MONEY[3]

- Klassisches Problem der Kryptographie
- Jeder Buchstabe repräsentiert eine Ziffer von 0 bis 9
- Keine Ziffer darf doppelt vorkommen

```
  S E N D
+ M O R E
---------
M O N E Y
```

- Welcher Buchstabe steht für welchen Wert?

---

[3] Mit Hilfe von (Mixed-)Integer-Programmierung lässt sich dieses Problem schnell lösen.

Lösung des Rätsels: SEND+MORE=MONEY

Initiale Idee:

- Jeder Buchstabe wird durch eine Variable repräsentiert, die auf den Wertebereich 0 bis 9 beschränkt ist.
- Die Gleichung (zum Optimieren) wird dann wie folgt dargestellt:

$$\begin{array}{rrrrrrrrr} & 1000 & \cdot S & +100 & \cdot E & +10 & \cdot N + & D & + \\ & 1000 & \cdot M & +100 & \cdot O & +10 & \cdot R + & E & = \\ 10000 \cdot M & +1000 & \cdot O & +100 & \cdot N & +10 & \cdot E + & Y & \end{array}$$

Ergebnis: Alle Variablen bekommen den Wert „0“ zugewiesen.

Herausforderung: Wir müssen die Nebenbedingungen formulieren, die sicherstellen, dass die Variablen die Werte 0 bis 9 annehmen und dass keine Ziffer doppelt vorkommt.

Problem: Es ist nicht direkt möglich eine mathematische Formulierung zu finden, die die Nebenbedingungen beschreibt.

Lösungsansatz (häufiger Ansatz bei „*Set-Partitioning-Problems*“):

- Jeder Variablen ([S, E, N, D, M, O, R, Y]) werden jeweils 10 binäre Variablen zugewiesen, die den Wert 0 oder 1 annehmen, wenn die Variable den entsprechenden Wert hat.

Nebenbedingungen

- Jede Variable hat genau einen Wert
- Keine Ziffer darf doppelt vorkommen

$$\begin{array}{ccccccccc} S_0 & + & S_1 & + & \dots & + & S_9 & = & 1 \\ + & & + & & \dots & & + & & \\ E_0 & + & E_1 & + & \dots & + & E_9 & = & 1 \\ \vdots & & \vdots & & \vdots & & \vdots & & \\ + & & + & & \dots & & + & & \\ Y_0 & + & Y_1 & + & \dots & + & Y_9 & = & 1 \\ || & & || & & & & || & & \\ 1 & & 1 & & \dots & & 1 & & \end{array}$$

„Optimierungsziel“

$$\begin{aligned} &\sum_{i=0}^{9} i \cdot S_i \times 1000 + \sum_{i=0}^{9} i \cdot E_i \times 100 + \sum_{i=0}^{9} i \cdot N_i \times 10 + \sum_{i=0}^{9} i \cdot D_i \times 1 \quad + \\ &\sum_{i=0}^{9} i \cdot M_i \times 1000 + \sum_{i=0}^{9} i \cdot O_i \times 100 + \sum_{i=0}^{9} i \cdot R_i \times 10 + \sum_{i=0}^{9} i \cdot E_i \times 1 \quad = \\ &\sum_{i=0}^{9} i \cdot M_i \times 10000 + \sum_{i=0}^{9} i \cdot O_i \times 1000 + \sum_{i=0}^{9} i \cdot N_i \times 100 + \sum_{i=0}^{9} i \cdot E_i \times 10 + \sum_{i=0}^{9} i \cdot Y_i \times 1 \end{aligned}$$

Umsetzung in Python mit Hilfe von `PuLP <https://coin-or.github.io/pulp/>`

Imports

```
from pulp import (
    LpProblem, LpVariable, LpBinary,
```

```
    lpSum
)
```

Variablen

```
VALS = range(10)
LETTERS = ["S", "E", "N", "D", "M", "O", "R", "Y"]

prob = LpProblem("SendMoreMoney") # Hier ist kein Optimierungsziel anzugeben.

choices = LpVariable.dicts("Choice", (LETTERS, VALS), cat=LpBinary)
```

Nebenbedingungen

```
# Jeder Buchstabe muss einen Wert haben
for l in LETTERS:
    varsOfLetter = [choices[l][i] for i in VALS]
    prob += lpSum(varsOfLetter) == 1

# Jeder Wert (0..9) darf nur einmal vorkommen
for i in VALS:
    varsOfValue = [choices[l][i] for l in LETTERS]
    prob += lpSum(varsOfValue) <= 1
```

„hauptsächliche Nebenbedingung“

```
prob += (
        lpSum([i*choices["S"][i] for i in range(10)]) * 1000
    +   lpSum([i*choices["E"][i] for i in range(10)]) * 100
    +   lpSum([i*choices["N"][i] for i in range(10)]) * 10
    +   lpSum([i*choices["D"][i] for i in range(10)])
    +   lpSum([i*choices["M"][i] for i in range(10)]) * 1000
    +   lpSum([i*choices["O"][i] for i in range(10)]) * 100
    +   lpSum([i*choices["R"][i] for i in range(10)]) * 10
    +   lpSum([i*choices["E"][i] for i in range(10)])
    == lpSum([i*choices["M"][i] for i in range(10)]) * 10000
    +   lpSum([i*choices["O"][i] for i in range(10)]) * 1000
    +   lpSum([i*choices["N"][i] for i in range(10)]) * 100
    +   lpSum([i*choices["E"][i] for i in range(10)]) * 10
    +   lpSum([i*choices["Y"][i] for i in range(10)]))
```

Lösung berechnen lassen

```
prob.solve()
values = [
    c + "=" + str(i)
        for c in choices
        for i in range(10)
            if choices[c][i].value() == 1
]
print("; ".join(values))
```

Ausgabe

```
S=8; E=3; N=2; D=4; M=0; O=9; R=1; Y=7
```

Code: https://delors.github.io/theo-algo-mixed_integer_programming/code/send_more_money.py

---

Eine Formulierung wie $28 \leq S + E + N + D + M + O + R + Y \leq 44$, um sicherzustellen, dass (zumindest einige) Variablen nicht 0 sind stellt leider nicht die gewünschte Nebenbedingung sicher, dass jeder Wert nur einmal vergeben wird ($0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 = 28$ und $0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45$).

Eine Lösung mit obiger Nebenbedingung wäre zum Beispiel:

```
    S=0 E=8 N=9 D=0
    M=0 O=0 R=9 E=8
M=0 O=0 N=9 E=8 Y=8
```

PuLP (Details)

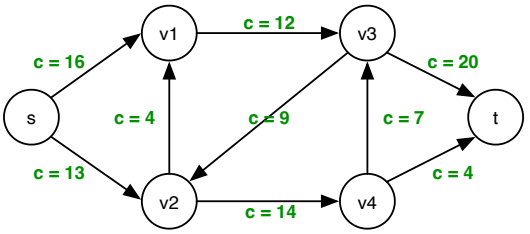
Die Datenstruktur `choices` ist ein Dictionary mit folgendem Aufbau:

```
choices =
    {'S': {0: Choice_S_0, 1: Choice_S_1,..., 9: Choice_S_9},
     ...
     'Y': {0: Choice_Y_0, 1: Choice_Y_1,..., 9: Choice_Y_9}}
# choices['S'][0].name == 'Choice_S_0'
```

✎ Übung

3.1. Maximaler Fluss

Berechnen Sie für folgenden Graphen den maximalen Fluss mit Hilfe von Pulp. Der Graph ist in der Vorlage definiert und kann als Grundlage für das Lösen des Problems verwendet werden. Orientieren Sie sich an dem Programm, dass sie im Vorfeld für das *Maximum-Flow-Problem* erstellt haben.



Vorlage

```
from pulp import (
    LpProblem,
    LpVariable,     # Erzeugt eine Variable
    LpMaximize,
    LpStatus,
    value,          # Gibt den Wert einer Variablen zurück
    lpSum,
)

CAPACITIES = {      # Kapazitäten der Kanten des Netzwerks
    "S": {"V1": 16, "V2": 13},
    "V1": {"V3": 12},
    "V2": {"V1": 4, "V4": 14},
    "V3": {"V2": 9, "T": 20},
    "V4": {"V3": 7, "T": 4},
    "T": {},
}

prob = LpProblem("Maximum Flow", LpMaximize)
```

✎ Übung

3.2. Gruppenzuteilung

Finden Sie eine sehr gute Aufteilung von Personen (Studierenden) auf eine feste Anzahl an Gruppen, basierend auf den Präferenzen der Personen. Nutzen Sie dazu Mixed-Integer-Programmierung. Im Template ist eine initiale Aufgabenstellung hinterlegt, die es zu lösen gilt: Verteilung von 16 Studierenden auf 4 Gruppen inkl. Bewertungsmatrix (jeder Studierende hat jeden anderen mit Werten von 1 bis 10 bewertet). Ggf. ist die Funktion `pulp.allcombinations` beim Modellieren hilfreich.

https://delors.github.io/theo-algo-mixed_integer_programming/code/group_assignment_template.py

3.3. Alle Gruppen gleich glücklich machen

Fragen Sie sich was Sie tun müssten, wenn Sie zusätzlich sicherstellen wollen, dass alle Gruppen in etwa die gleiche Glücklichkeit haben sollen. (Hier geht es nur um ein Gedankenexperiment.)

Klausurvorbereitung

- Finden Sie eine Formulierung für das Lösen von Sudokus mittels Mixed-Integer-Programmierung.

Nächste Schritte

- Studiere mathematische Programmiersprachen (AMPL, ZIMPL).
- Studiere verfügbare Bibliotheken zum Lösen entsprechender Probleme (GLPK, SCIP, Gurobi, CPLEX, ...)

Hinweis

PuLP ist ein einfaches, aber mächtiges Werkzeug, um lineare und gemischt-ganzzahlige Programme zu beschreiben. PuLP nutzt im Hintergrund verschiedene Solver!